

Database Schema DatabaseDesign(DDL with Constraints & Auto-Increment)

```
KD2_98846_Raj>CREATE DATABASE cdac_admission;
Query OK, 1 row affected (0.14 sec)

KD2_98846_Raj>USE cdac_admission;
Database changed
KD2_98846_Raj>--1.CENTERS TABLE
-> -- 1.CENTERS TABLE
-> --1.CENTERS TABLE
-> -- 1.CENTERS TABLE
-> ^C
KD2_98846_Raj> -- 1.CENTERS TABLE
-> # CENTERS TABLE
-> -- 1.CENTERS TABLE
-> ^C
KD2_98846_Raj> -- 1.CENTER TABLE
-> -- 1.CENTER TABLE;
-> ^C
KD2_98846_Raj> -- 1.CENTER TABLE
-> CREATE TABLE centres (
->     centre_id INT AUTO_INCREMENT,
->     centre_name VARCHAR(100) NOT NULL,
->     city VARCHAR(50) NOT NULL,
->     CONSTRAINT pk_centres PRIMARY KEY (centre_id),
->     CONSTRAINT uq_centre_name UNIQUE (centre_name)
-> );
Query OK, 0 rows affected (0.32 sec)
```

```

KD2_98846_Raj>
KD2_98846_Raj>-- 2. COURSES TABLE
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>
KD2_98846_Raj>-- 2. COURSES TABLE
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>CREATE TABLE courses (
->     course_id INT AUTO_INCREMENT,
->     course_name VARCHAR(50) NOT NULL,
->     duration_months INT DEFAULT 6,
->     CONSTRAINT pk_courses PRIMARY KEY (course_id),
->     CONSTRAINT uq_course_name UNIQUE (course_name),
->     CONSTRAINT chk_duration CHECK (duration_months > 0)
-> );
Query OK, 0 rows affected (0.10 sec)

KD2_98846_Raj>-- 3. CENTRE-COURSE MAPPING (Composite Key Example)
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>CREATE TABLE centre_courses (
->     centre_id INT,
->     course_id INT,
->     capacity INT NOT NULL,
->     seats_filled INT DEFAULT 0,
->     CONSTRAINT pk_centre_courses
->         PRIMARY KEY (centre_id, course_id),
->
->     CONSTRAINT fk_cc_centre
->         FOREIGN KEY (centre_id)
->         REFERENCES centres(centre_id)
->         ON DELETE CASCADE,
->
->     CONSTRAINT fk_cc_course
->         FOREIGN KEY (course_id)
->         REFERENCES courses(course_id)
->         ON DELETE CASCADE,
->
->     CONSTRAINT chk_capacity
->         CHECK (capacity >= 0),
->
->     CONSTRAINT chk_seats_filled
->         CHECK (seats_filled <= capacity)
-> );
Query OK, 0 rows affected (0.11 sec)

```

```
KD2_98846_Raj>-- 4. CANDIDATES TABLE  
Query OK, 0 rows affected (0.00 sec)
```

```
KD2_98846_Raj>CREATE TABLE candidates (  
->   candidate_id INT AUTO_INCREMENT,  
->   name VARCHAR(100) NOT NULL,  
->   email VARCHAR(100) NOT NULL,  
->   ccat_rank INT NOT NULL,  
->   allocated_centre_id INT NULL,  
->   allocated_course_id INT NULL,  
->   allocation_status VARCHAR(20) DEFAULT 'NOT_ALLOCATED',  
->  
->   CONSTRAINT pk_candidates  
->     PRIMARY KEY (candidate_id),  
->  
->   CONSTRAINT uq_candidate_email  
->     UNIQUE (email),  
->  
->   CONSTRAINT uq_candidate_rank  
->     UNIQUE (ccat_rank),  
->  
->   CONSTRAINT fk_cand_allocated  
->     FOREIGN KEY (allocated_centre_id, allocated_course_id)  
->     REFERENCES centre_courses(centre_id, course_id),  
->  
->   CONSTRAINT chk_rank  
->     CHECK (ccat_rank > 0),  
->  
->   CONSTRAINT chk_status  
->     CHECK (  
->       allocation_status IN  
->       ('NOT_ALLOCATED', 'ROUND_1', 'ROUND_2', 'CONFIRMED')  
->     )  
-> );  
Query OK, 0 rows affected (0.22 sec)
```

```
KD2_98846_Raj>-- 5. PREFERENCES TABLE (Composite Key Example)
Query OK, 0 rows affected (0.00 sec)
```

```
KD2_98846_Raj>CREATE TABLE preferences (
->   candidate_id INT,
->   preference_number INT,
->   centre_id INT,
->   course_id INT,
->
->   CONSTRAINT pk_preferences
->     PRIMARY KEY (candidate_id, preference_number),
->
->   CONSTRAINT fk_pref_candidate
->     FOREIGN KEY (candidate_id)
->     REFERENCES candidates(candidate_id)
->     ON DELETE CASCADE,
->
->   CONSTRAINT fk_pref_cc
->     FOREIGN KEY (centre_id, course_id)
->     REFERENCES centre_courses(centre_id, course_id),
->
->   CONSTRAINT chk_pref_num
->     CHECK (preference_number > 0)
-> );
Query OK, 0 rows affected (0.15 sec)
```

Sample Records Insertion (DML)

```

Command Prompt - mysql -u x + v

KD2_98846_Raj>-- Insert Candidates (Ordered by performance)
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>
KD2_98846_Raj>INSERT INTO candidates (name, email, ccat_rank) VALUES
-> ('Amit Sharma', 'amit@gmail.com', 1),
-> ('Priya Patel', 'priya@gmail.com', 2),
-> ('Rahul Verma', 'rahul@gmail.com', 3),
-> ('Sneha Reddy', 'sneha@gmail.com', 4);
ERROR 1062 (23000): Duplicate entry 'amit@gmail.com' for key 'candidates.uq_candidate_email'

KD2_98846_Raj>-- Priya (Rank 2): Prefers IACSD DAC, then IACSD DBDA
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>
KD2_98846_Raj>INSERT INTO preferences VALUES
-> (2, 1, 1, 1),
-> (2, 2, 1, 2);
Query OK, 2 rows affected (0.06 sec)
Records: 2 Duplicates: 0 Warnings: 0

KD2_98846_Raj>-- Insert Preferences for Candidates
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>
KD2_98846_Raj>-- Amit (Rank 1): Prefers IACSD DAC, then Acts DAC
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>
KD2_98846_Raj>INSERT INTO preferences VALUES
-> (1, 1, 1, 1),
-> (1, 2, 2, 1);
Query OK, 2 rows affected (0.05 sec)
Records: 2 Duplicates: 0 Warnings: 0

KD2_98846_Raj>-- Insert Preferences for Candidates
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>
KD2_98846_Raj>-- Amit (Rank 1): Prefers IACSD DAC, then Acts DAC
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>-- Sneha (Rank 4): Prefers IACSD DBDA, then Sunbeam DAC
Query OK, 0 rows affected (0.00 sec)

KD2_98846_Raj>
KD2_98846_Raj>INSERT INTO preferences VALUES
-> (4, 1, 1, 2),
-> (4, 2, 3, 1);
Query OK, 2 rows affected (0.05 sec)
Records: 2 Duplicates: 0 Warnings: 0

```

Stored Procedure for 1 Round Allocation

```
Command Prompt - mysql -u x + v
KD2_98846_Raj>DELIMITER $$
KD2_98846_Raj>
KD2_98846_Raj>CREATE PROCEDURE ProcessRound1Allocation(OUT out_msg VARCHAR(255))
-> BEGIN
->
->     -- Variables for Candidate Cursor
->     DECLARE v_cand_id INT;
->     DECLARE v_rank INT;
->     DECLARE cand_done INT DEFAULT 0;
->
->     -- Variables for Preference Processing
->     DECLARE v_pref_centre INT;
->     DECLARE v_pref_course INT;
->     DECLARE v_capacity INT;
->     DECLARE v_filled INT;
->     DECLARE pref_done INT DEFAULT 0;
->
->     -- Candidate Cursor sorted by best Rank
->     DECLARE cand_cursor CURSOR FOR
->         SELECT candidate_id, ccat_rank FROM candidates ORDER BY ccat_rank ASC;
->
->     -- Declaring handlers for NOT FOUND
->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET cand_done = 1;
->
->     -- Robust Error Handling
->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
->     BEGIN
->         ROLLBACK;
->         SET out_msg = 'ERROR: Round 1 allocation failed due to a database exception.';
->     END;
->
->     START TRANSACTION;
->
->     OPEN cand_cursor;
->
->     cand_loop: LOOP
->
->         FETCH cand_cursor INTO v_cand_id, v_rank;
->
->         IF cand_done = 1 THEN
->             LEAVE cand_loop;
->         END IF;
->
->         -- Nested block to process preferences for the fetched candidate
->         BEGIN
->
->             DECLARE v_pref_num INT DEFAULT 1;

```

```
Command Prompt - mysql -u X + v
-> DECLARE pref_cursor CURSOR FOR
->     SELECT centre_id, course_id FROM preferences
->     WHERE candidate_id = v_cand_id
->     ORDER BY preference_number ASC;
->
-> DECLARE CONTINUE HANDLER FOR NOT FOUND SET pref_done = 1;
->
-> SET pref_done = 0;
->
-> OPEN pref_cursor;
->
-> pref_loop: LOOP
->
->     FETCH pref_cursor INTO v_pref_centre, v_pref_course;
->
->     IF pref_done = 1 THEN
->         LEAVE pref_loop;
->     END IF;
->
->     -- Check seat availability
->     SELECT capacity, seats_filled INTO v_capacity, v_filled
->     FROM centre_courses
->     WHERE centre_id = v_pref_centre
->           AND course_id = v_pref_course;
->
->     -- Control structure: Check if seat can be allocated
->     IF v_filled < v_capacity THEN
->
->         -- Allocate Seat
->         UPDATE candidates
->         SET allocated_centre_id = v_pref_centre,
->             allocated_course_id = v_pref_course,
->             allocation_status = 'ROUND_1'
->         WHERE candidate_id = v_cand_id;
->
->         -- Increment filled seats counter
->         UPDATE centre_courses
->         SET seats_filled = seats_filled + 1
->         WHERE centre_id = v_pref_centre
->               AND course_id = v_pref_course;
->
->         -- Allocation successful, break preference loop
->         SET pref_done = 1;
->
->     END IF;
->
-> END LOOP pref_loop;
->
-> CLOSE pref_cursor;
```

```
->
->         CLOSE pref_cursor;
->
->         END;
->
->     END LOOP cand_loop;
->
->     CLOSE cand_cursor;
->
->     COMMIT;
->
->     SET out_msg = 'SUCCESS: Round 1 allocation completed smoothly.';
->
-> END$$
Query OK, 0 rows affected (0.03 sec)
```

```
Command Prompt - mysql -u X + v
->         SELECT preference_number
->         FROM preferences
->         WHERE candidate_id = v_cand_id
->             AND centre_id = v_current_centre
->             AND course_id = v_current_course
->     )
->     ORDER BY preference_number ASC;
->
->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET pref_done = 1;
->
->     SET pref_done = 0;
->
->     OPEN pref_cursor;
->
->     pref_loop: LOOP
->
->         FETCH pref_cursor INTO v_pref_centre, v_pref_course;
->
->         IF pref_done = 1 THEN
->             LEAVE pref_loop;
->         END IF;
->
->         -- Check if higher preference choice has space
->         SELECT capacity, seats_filled INTO v_capacity, v_filled
->         FROM centre_courses
->         WHERE centre_id = v_pref_centre
->             AND course_id = v_pref_course;
->
->         IF v_filled < v_capacity THEN
->
->             -- Free up previous allocation seat (if they had one)
->             IF v_current_centre IS NOT NULL THEN
->
->                 UPDATE centre_courses
->                 SET seats_filled = seats_filled - 1
->                 WHERE centre_id = v_current_centre
->                     AND course_id = v_current_course;
->
->             END IF;
->
->             -- Allocate new upgraded seat
->             UPDATE candidates
->             SET allocated_centre_id = v_pref_centre,
->                 allocated_course_id = v_pref_course,
->                 allocation_status = 'ROUND_2'
->             WHERE candidate_id = v_cand_id;
->
->             -- Occupy new seat
->             UPDATE centre_courses
```

Stored Procedure for Round 2 Allocation

```
Command Prompt - mysql -u X + v
D2_98846_Raj>
D2_98846_Raj>DELIMITER ;
D2_98846_Raj>DELIMITER $$
D2_98846_Raj>
D2_98846_Raj>CREATE PROCEDURE ProcessRound2Allocation(OUT out_msg VARCHAR(255))
-> BEGIN
->
->     DECLARE v_cand_id INT;
->     DECLARE v_current_centre INT;
->     DECLARE v_current_course INT;
->     DECLARE cand_done INT DEFAULT 0;
->
->     DECLARE v_pref_centre INT;
->     DECLARE v_pref_course INT;
->     DECLARE v_capacity INT;
->     DECLARE v_filled INT;
->     DECLARE pref_done INT DEFAULT 0;
->
->     -- Cursor to evaluate candidates again in order of rank
->     DECLARE cand_cursor CURSOR FOR
->         SELECT candidate_id, allocated_centre_id, allocated_course_id
->         FROM candidates ORDER BY ccat_rank ASC;
->
->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET cand_done = 1;
->
->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
->     BEGIN
->         ROLLBACK;
->         SET out_msg = 'ERROR: Round 2 allocation aborted due to database fault.';
->     END;
->
->     START TRANSACTION;
->
->     OPEN cand_cursor;
->
->     cand_loop: LOOP
->
->         FETCH cand_cursor INTO v_cand_id, v_current_centre, v_current_course;
->
->         IF cand_done = 1 THEN
->             LEAVE cand_loop;
->         END IF;
->
->         BEGIN
->
->             -- Fetch only preferences that are HIGHER than the current allocation
->             -- If not allocated, fetches all preferences
->             DECLARE pref_cursor CURSOR FOR
```

e

```
->         AND course_id = v_pref_course;
->
->         SET pref_done = 1;
->
->     END IF;
->
-> END LOOP pref_loop;
->
-> CLOSE pref_cursor;
->
-> END;
->
-> END LOOP cand_loop;
->
-> CLOSE cand_cursor;
->
-> COMMIT;
->
-> SET out_msg = 'SUCCESS: Round 2 allocations/upgradations completed.';
->
-> END$$
Query OK, 0 rows affected (0.06 sec)
```

Testing the Allocations -

```
KD2_98846_Raj>
KD2_98846_Raj>DELIMITER ;
KD2_98846_Raj>CALL ProcessRound1Allocation(@message1);
Query OK, 0 rows affected (0.09 sec)
```

```
KD2_98846_Raj>SELECT @message1;
```

```
+-----+
| @message1 |
+-----+
| SUCCESS: Round 1 allocation completed smoothly. |
+-----+
1 row in set (0.00 sec)
```

```
KD2_98846_Raj>SELECT name, ccat_rank, allocated_centre_id, allocation_status
-> FROM candidates;
```

```
+-----+-----+-----+-----+
| name      | ccat_rank | allocated_centre_id | allocation_status |
+-----+-----+-----+-----+
| Amit Sharma | 1         | 1                   | ROUND_1           |
| Priya Patel | 2         | 1                   | ROUND_1           |
| Rahul Verma | 3         | 2                   | ROUND_1           |
| Sneha Reddy | 4         | 1                   | ROUND_1           |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
KD2_98846_Raj>CALL ProcessRound2Allocation(@message2);
Query OK, 0 rows affected (0.05 sec)
```

```
KD2_98846_Raj>SELECT @message2;
```

```
+-----+
| @message2 |
+-----+
| SUCCESS: Round 2 allocations/upgradations completed. |
+-----+
1 row in set (0.00 sec)
```

```
KD2_98846_Raj>SELECT name, ccat_rank, allocated_centre_id, allocation_status
-> FROM candidates;
```

```
+-----+-----+-----+-----+
| name      | ccat_rank | allocated_centre_id | allocation_status |
+-----+-----+-----+-----+
| Amit Sharma | 1         | 1                   | ROUND_1           |
| Priya Patel | 2         | 1                   | ROUND_1           |
| Rahul Verma | 3         | 2                   | ROUND_1           |
| Sneha Reddy | 4         | 1                   | ROUND_1           |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```